Trace | High-Level Design

MVP scope and architectural direction

1. MVP Scope

- Automatic location: Request high-accuracy device GPS when the app opens, reuse permission when available, show accuracy and a clear retry option.

- Round trips: v0.1 generates out-and-back routes that start and end at the detected location. Loop generation is deferred.

- Route generation: Default to 5 km; request 1–3 routes. Aim for ±5% distance accuracy when feasible, validate pedestrian suitability, and avoid near-duplicates. Accuracy and suitability take priority over variety; one or two valid routes are a successful result.

- UX and speed: Aim for p95 under 5 seconds for generation (excluding initial map load); use cellular data or Wi-Fi. Display a useful error if GPS, connectivity, or routing fails.

2. Selected Architecture

React Native + Expo (TypeScript) -> Cloudflare Worker (TypeScript) -> GraphHopper routing API (OpenStreetMap network).
Mobile: Expo Location obtains GPS coordinates; map UI shows 1-3 options; persist generated route geometry locally for later viewing. Offline basemaps require separate tile caching.
Worker: POST /api/v1/routes/generate validates inputs, samples candidate turnaround points, requests pedestrian paths, and evaluates and ranks results. Store the GraphHopper API key as a Worker secret.
Routing: GraphHopper's hosted standard point-to-point pedestrian routing over OpenStreetMap. Trace supplies distance-targeting logic; v0.1 does not depend on the provider's round_trip generator. No database or always-on personal computer is required.
Cost: Target $0/month for personal use. Bound GraphHopper calls and concurrency; stop when enough valid routes are found or the request budget is exhausted. Confirm plan quotas, supported routing modes and map tile licensing before deployment.

3. Other Architectures Considered

Python FastAPI + OSMnx/NetworkX on hosted server: Maximum algorithm control and local graph caching, but higher operations cost, dataset management, and possible free-host cold starts.

React PWA + hosted routing API: Fastest cross-platform launch and low maintenance, but less native mobile control and less reliable background capabilities.
Fully on-device/offline routing: Excellent no-signal experience and no per-request API usage, but complex map downloads, storage, and mobile routing engine integration.

4. Future Roadmap

Phase 1: Working mobile flow with GPS, 5 km default, 1-3 out-and-back routes, route map, and distance validation.

Phase 2: Loop routes, distinctness scoring, route regeneration, parks/trails preference, and performance measurement on cellular.

Phase 3: GPX export, favorites, offline route/map caching, elevation and safety-aware preferences. Revisit Dijkstra-based exploration after a usable MVP; it requires access to a pedestrian network graph.

5. Finalized Decisions

The four architectural decisions are complete. Implementation is next.

Step	Decision	Agreed decision
1	Routing engine	GraphHopper hosted pedestrian routing; Trace handles distance targeting.
2	Route generation algorithm	Sample turnaround candidates in multiple directions using geographic distance (roughly 1.5–2.5 km away for a 5 km run). Ask GraphHopper for actual pedestrian paths, choose or trim a valid turnaround near half the target distance, and reverse the path for the return. Check return suitability and rank by distance accuracy and pedestrian suitability. Return up to the requested count; exact distance is not guaranteed.
3	Infrastructure	React Native + Expo and a TypeScript Cloudflare Worker; route geometry stored locally. GraphHopper is the expected

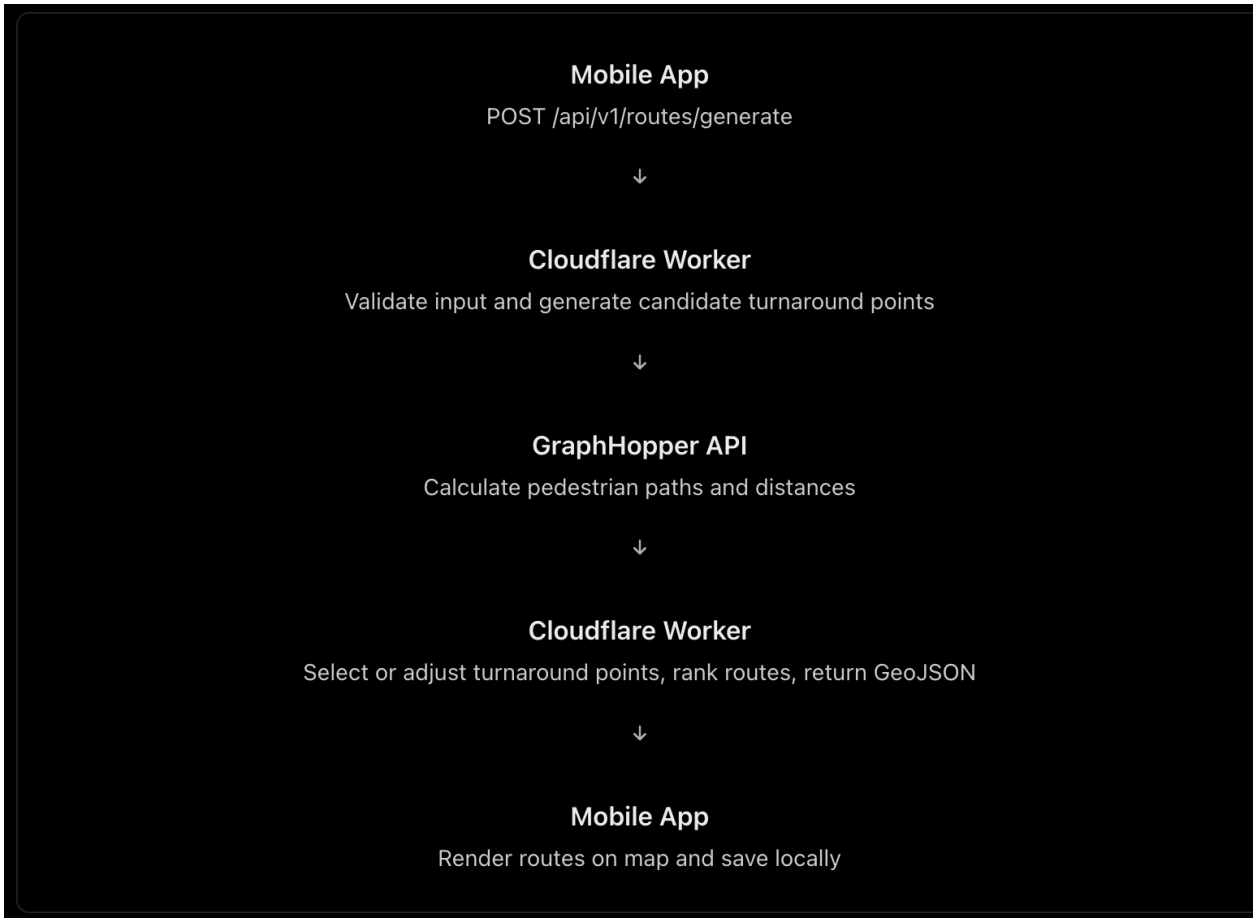
		bottleneck. Planning estimate: 500 daily credits ÷ 8 one-credit calls per generation ≈ 62 route sets/day; this is an assumption to verify, not a guaranteed quota or fixed call budget.
4	API design	One versioned REST endpoint, meters internally, GeoJSON geometry, and best-effort delivery of 1–3 routes. See the API contract below.

6. API Contract

POST /api/v1/routes/generate
Request: latitude and longitude (numbers), distanceMeters (positive integer), and routeCount (integer, 1–3).
Response: Each route includes an ID, out-and-back type, actual distance, and complete outbound-and-return GeoJSON geometry.
Success: 200 OK when at least one valid route is found, even if fewer than requested are returned. Zero valid routes: 422 with NO_ROUTES_FOUND.
Other errors: 400 invalid input; 429 rate/quota limit; 502 routing-provider failure; 504 routing timeout. Use consistent error responses.

7. Implementation Next Steps

Use the existing GitHub repository, currently containing a README. Proposed layout: backend/ for the Worker and mobile/ for React Native + Expo, alongside README.md and .gitignore.
Build in order: TypeScript Worker → GraphHopper pedestrian integration → candidate generation and distance validation → mobile GPS, route map and API integration. Test the backend independently before connecting the mobile app.


Mobile App
POST /api/v1/routes/generate
↓
Cloudflare Worker
Validate input and generate candidate turnaround points
↓
GraphHopper API
Calculate pedestrian paths and distances
↓
Cloudflare Worker
Select or adjust turnaround points, rank routes, return GeoJSON
↓
Mobile App
Render routes on map and save locally